

Visões, Linguagem SQL e Indexação

Unidade 3 · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Nesta aula

- **Visões (Views)** — o que são, objetivos e limites
- A **Linguagem SQL** — história e os grupos **DDL, DML, DQL, DCL**
- **Indexação** — índices ordenados (esparso/denso) e hash

Visões (Views)

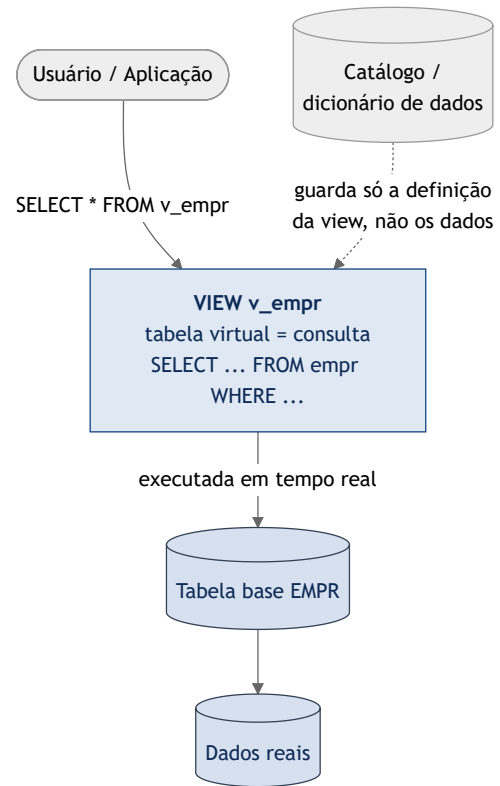
O que é uma View

- Uma **tabela virtual** definida por uma **consulta** — não armazena dados próprios (salvo *materialized views*).
- Formas diferentes de **visualizar** o mesmo conjunto de dados.

Objetivos:

- **Segurança** — expor só colunas/linhas permitidas.
- **Simplificação** — esconder consultas complexas atrás de um nome.
- **Reúso** — "armazenar" consultas complexas.

Estrutura de uma View



A view guarda **só a definição** (no catálogo); os dados continuam nas **tabelas base**.

Definindo Views

```
CREATE VIEW alunos_matriculados AS
SELECT a.matricula, a.nome, o.disciplina
FROM alunos a
JOIN alunos_ofertas ao ON ao.aluno = a.matricula
JOIN ofertas o ON o.codigo = ao.oferta;
```

Operações comuns na definição: `WHERE` (filtrar linhas), escolher **colunas**, `GROUP BY` (sumarizar), `JOIN` (múltiplas tabelas).

Views atualizáveis

Uma view é **atualizável** (aceita `INSERT/UPDATE/DELETE`) quando, em geral:

- Deriva de **uma única** tabela/consulta;
- Inclui as **chaves primárias** e as colunas **não nulas** obrigatórias;
- **Não** usa `DISTINCT` , `GROUP BY` , agregações ou remove duplicatas.

Views com junção/agregação normalmente são **somente leitura** — para alterar dados, use as **tabelas base**.

A Linguagem SQL

História e padrões

- **SQL** (*Structured Query Language*): linguagem **declarativa** — você descreve **o que** quer, não **como** obter.
- Origem: **SEQUEL** (*Structured English Query Language*), IBM System R.
- Padrões **ANSI/ISO**: SQL-86, SQL-89, **SQL-92 (SQL2)**, SQL:1999, SQL:2003 ... SQL:2016/2023.

💡 Os SGBDs não implementam 100% do padrão e ainda acrescentam extensões próprias — daí as diferenças entre Oracle, PostgreSQL e MySQL.

Os quatro grupos da SQL

Grupo	Para quê	Comandos
DDL — <i>Definition</i>	estrutura	CREATE , ALTER , DROP
DML — <i>Manipulation</i>	dados	INSERT , UPDATE , DELETE
DQL — <i>Query</i>	consulta	SELECT
DCL — <i>Control</i>	permissões	GRANT , REVOKE

(alguns autores incluem *TCL* — COMMIT , ROLLBACK , SAVEPOINT .)



A prática completa de DDL/DML/DQL está no **Roteiro Prático de SQL** (Unidade 5), na VM.

Indexação

Índices

Estrutura **adicional** associada a uma tabela para **acelerar consultas** — como o índice remissivo de um livro.

- **Vantagem:** buscas muito mais rápidas (evita varredura completa).
- **Custo:** ocupa espaço e **torna escritas** (INSERT/UPDATE/DELETE) um pouco mais lentas (o índice também é mantido).

```
CREATE INDEX alunos_nome_idx ON alunos(nome);
```

Índices Ordenados: esperso × denso

- **Denso:** uma entrada de índice para **cada** valor de chave da tabela.
- **Esperso:** uma entrada apenas para **alguns** valores (ex.: um por bloco) — menor, mas exige leitura sequencial a partir da entrada.

Tipo	Entradas	Tamanho	Busca
Denso	todas as chaves	maior	direta
Esperso	algumas chaves	menor	aproxima + varre

Índices Multiníveis

- Se o próprio índice fica **grande demais** para varrer rápido, cria-se um **índice sobre o índice**.
- O nível de cima (esparso) aponta para **blocos** do nível de baixo → forma uma **árvore** de níveis.
- Reduz a busca de **linear** (varrer o índice) para **logarítmica** — poucos acessos a disco.

💡 É a ideia por trás dos índices **B-tree / B+-tree**, o tipo padrão na maioria dos SGBDs.

Índices Hash

- Aplicam uma **função hash** à chave para localizar o registro em **tempo praticamente constante**.
- Ótimos para busca por **igualdade** (`= valor`); **ruins** para faixas (`BETWEEN`, `>`, `ORDER BY`).

💡 Regra prática: indexe as colunas usadas em `WHERE`, `JOIN` e `ORDER BY` frequentes — mas não indexe tudo (custo de escrita).

Exercícios de Consolidação

Unidade 3 · 12 questões estilo ENADE

Modelo Relacional · Álgebra · Views/SQL/Indexação — responda antes de virar o slide!

Q1 — Modelo Relacional: chave primária e estrangeira

No sistema acadêmico há `TURMA(id_turma, nome)` e `ALUNO(matricula, nome, id_turma)`, em que **cada aluno pertence a uma turma**.

Assinale a alternativa correta sobre as chaves dessas tabelas.

- A) `matricula` é a **chave primária** de `ALUNO` e a identifica unicamente; `id_turma`, em `ALUNO`, é **chave estrangeira** que referencia a PK de `TURMA`.
- B) `id_turma`, em `ALUNO`, é a **chave primária** de `ALUNO`.
- C) A chave primária **pode conter nulos**, desde que a maioria das linhas esteja preenchida.
- D) A chave estrangeira precisa ter o **mesmo nome** da chave primária que referencia.

Q1 — Gabarito: A

Por que A: a **chave primária** identifica unicamente cada tupla e **não admite nulos** (`matricula` em `ALUNO`). A **chave estrangeira** guarda valores que **existem como PK** na tabela referenciada (`id_turma` → `TURMA`), garantindo a integridade referencial.

- **B** — errada: `id_turma`, em `ALUNO`, **aponta para** `TURMA` (é FK); a PK de `ALUNO` é `matricula`.
- **C** — errada: PK **nunca** admite nulos (integridade de entidade).
- **D** — errada: o **nome** não precisa coincidir; o que importa é referenciar a **PK correta**, com valores/domínio compatíveis.

Q2 — Modelo Relacional: integridade referencial

Em `PEDIDO(id, id_cliente)`, com `id_cliente` referenciando `CLIENTE`, tenta-se **excluir** um cliente que **ainda possui pedidos**.

Para o banco **impedir** a exclusão e manter a consistência, o conceito/mecanismo aplicável é:

- A) Integridade de **domínio**, via `CHECK`.
- B) Integridade **referencial**, com ação `ON DELETE RESTRICT` / `NO ACTION`.
- C) Integridade de **entidade**, garantida pela PK de `PEDIDO`.
- D) Nenhuma; o SGBD sempre permite excluir.

Q2 — Gabarito: B

Por que B: a **integridade referencial** garante que toda FK aponte para uma linha existente; `RESTRICT / NO ACTION` **bloqueia** a exclusão do cliente enquanto houver pedidos referenciando-o.

- A — errada: domínio trata de **valores válidos numa coluna** (tipo/faixa), não do vínculo entre tabelas.
- C — errada: integridade de **entidade** é sobre a PK (não nula/única), não sobre a relação entre `PEDIDO` e `CLIENTE`.
- D — errada: o SGBD justamente **impede** exclusões que quebrem a FK.

Q3 — Modelo Relacional: propriedades da relação

No modelo relacional de **Codd**, uma relação é definida a partir da **teoria dos conjuntos**.

Sobre as propriedades de uma **relação (tabela)**, assinale a alternativa **correta**.

- A) A **ordem das linhas** é significativa e deve ser preservada.
- B) Podem existir **duas tuplas idênticas** na mesma relação.
- C) Uma relação é um **conjunto**: a ordem das linhas não importa e não há tuplas duplicadas.
- D) Cada célula pode conter **vários valores** (atributo multivalorado) sem restrição.

Q3 — Gabarito: C

Por que C: como a relação é um **conjunto** de tuplas, **não há ordem** entre as linhas e **não existem tuplas duplicadas** (a chave primária garante a **unicidade**).

- **A** — errada: a **ordem das linhas é irrelevante** no modelo relacional.
- **B** — errada: **não** pode haver duas tuplas totalmente iguais (princípio da unicidade).
- **D** — errada: os valores devem ser **atômicos** (1FN) — atributo multivalorado não é permitido.

Q4 — Modelo Relacional: integridade de entidade

Tenta-se **inserir** um produto **sem preencher** a coluna que é **chave primária** (deixando-a **NULL**).

O que ocorre e por quê?

- A) É aceito; a PK pode ser nula se for a única linha.
- B) É **rejeitado**: a **integridade de entidade** exige PK **não nula e única**.
- C) É aceito; **NULL** conta como um valor único.
- D) É rejeitado por violar a integridade de **domínio**.

Q4 — Gabarito: B

Por que B: a **integridade de entidade** determina que a **chave primária** nunca seja nula e seja única — sem isso a linha não pode ser identificada.

- **A e C** — erradas: PK **nunca** é nula; e `NULL` **não** é um valor comparável (`NULL` não é igual a `NULL`), então não serve para identificar.
- **D** — errada: a regra ferida é de **entidade** (PK), não de **domínio** (valores válidos de uma coluna comum).

Q5 — Álgebra Relacional: seleção × projeção

Em `ALUNO(matricula, nome, curso)`, deseja-se obter **apenas os nomes** dos alunos do curso 'SI'.

Qual expressão da álgebra relacional está correta?

- A) $\pi_{\text{nome}}(\sigma_{\text{curso}='SI'}(\text{ALUNO}))$
- B) $\sigma_{\text{nome}}(\pi_{\text{curso}='SI'}(\text{ALUNO}))$
- C) $\sigma_{\text{curso}='SI'}(\text{ALUNO})$
- D) $\pi_{\text{curso}}(\sigma_{\text{nome}='SI'}(\text{ALUNO}))$

Q5 — Gabarito: A

Por que A: σ (seleção) filtra **linhas** pela condição `curso='SI'` ; π (projeção) mantém só a **coluna** `nome` .

- B — errada: troca os operadores — σ precisa de **condição** e π de **lista de colunas**.
- C — errada: filtra certo, mas devolve a **tupla inteira**, não só o `nome` .
- D — errada: projeta a coluna errada (`curso`) e filtra pelo campo errado (`nome`).

Q6 — Álgebra Relacional: junção natural

DEPTO tem **3 linhas** e FUNC tem **7 linhas**, cada funcionário com um `id_depto` **válido**.

Quantas linhas resultam de `DEPTO ⋈ FUNC` (junção natural por `id_depto`)?

- A) 21 linhas.
- B) 3 linhas.
- C) 7 linhas.
- D) 10 linhas.

Q6 — Gabarito: C

Por que C: cada linha de `FUNC` casa com **exatamente um** `DEPTO` (a FK sempre existe). A junção tem, então, o número de linhas do lado "**muitos**" = 7.

- **A** — errada: $21 = 3 \times 7$ é o **produto cartesiano**, que a junção **não** faz.
- **B** — errada: 3 é a quantidade de departamentos.
- **D** — errada: 10 é a **soma** $3 + 7$, sem sentido aqui.

Q7 — Álgebra Relacional: divisão

`CURSO(aluno, disciplina)` registra o que cada aluno cursou. Deseja-se **os alunos que cursaram TODAS** as disciplinas obrigatórias.

Qual operação expressa isso **naturalmente**?

- A) Junção ($\bowtie$).
- B) Divisão ($\div$).
- C) Interseção ($\cap$).
- D) Produto cartesiano ($\times$).

Q7 — Gabarito: B

Por que B: a **divisão** responde exatamente à ideia de "*quem se relaciona com **TODOS** os valores*" de um conjunto divisor (todas as disciplinas obrigatórias).

- **A** — errada: a junção apenas **combina** tabelas, não exige "todos".
- **C** — errada: interseção compara **relações compatíveis**, não "aluno × conjunto de disciplinas".
- **D** — errada: o produto cartesiano gera **todas as combinações**, sem o critério "todas".

Q8 — Álgebra Relacional: operações de conjunto

Deseja-se a **união** de duas relações R e S ($R \cup S$).

Para essa operação ser **válida**, é necessário que:

- A) R e S tenham o **mesmo número de linhas**.
- B) R e S sejam **compatíveis**: mesmo número de atributos e **domínios correspondentes**.
- C) R e S tenham a **mesma chave primária**.
- D) R e S estejam na **mesma tabela física**.

Q8 — Gabarito: B

Por que B: $\cup$, $\cap$ e $-$ exigem **compatibilidade de união**: mesma **quantidade de atributos** e **domínios correspondentes** (colunas na mesma ordem e tipo).

- A — errada: o número de **linhas** é irrelevante.
- C — errada: não é preciso mesma PK, e sim mesma **estrutura de colunas**.
- D — errada: são relações; não importa onde estão armazenadas.

Q9 — SQL: os grupos de comandos

A SQL organiza seus comandos em **grupos**, conforme a finalidade. Uma equipe precisa: (1) **criar a estrutura** de uma tabela; (2) **inserir e atualizar** registros; (3) **conceder permissões** de acesso.

Esses três objetivos pertencem, respectivamente, aos grupos:

- A) DML, DDL, DCL.
- B) DQL, DML, DDL.
- C) DDL, DML, DCL.
- D) DDL, DQL, DCL.

Q9 — Gabarito: C

Por que **C**: criar estrutura = **DDL** (CREATE/ALTER/DROP); inserir/atualizar dados = **DML** (INSERT/UPDATE/DELETE); conceder permissões = **DCL** (GRANT/REVOKE).

- **A** — errada: inverte DDL e DML — **criar a estrutura não é DML**.
- **B** — errada: embaralha tudo; **DQL** é apenas consulta (SELECT).
- **D** — errada: **inserir/atualizar dados não é DQL** (consulta).

Q10 — SQL: linguagem declarativa

Ao pedir dados ao banco, o profissional informa **quais** dados quer; o próprio SGBD decide **como** obtê-los (quais índices usar, ordem de acesso etc.).

Essa característica classifica a SQL como uma linguagem:

- A) **declarativa** — descreve-se o "**o quê**", não o "**como**".
- B) procedural — o programador descreve o **passo a passo** do acesso.
- C) de **baixo nível**, dependente do armazenamento físico.
- D) **apenas** de definição de estrutura (DDL).

Q10 — Gabarito: A

Por que A: em SQL você **declara o que** deseja e o SGBD determina **como** buscar — por isso é **declarativa** e de **alto nível**.

- **B** — errada: seria o oposto — na abordagem **procedural** você especificaria o algoritmo de acesso (o "como").
- **C** — errada: SQL é de **alto nível** e **independe** do armazenamento físico (independência de dados).
- **D** — errada: a SQL tem **vários grupos** (DDL, DML, DQL, DCL), não só definição de estrutura.

Q11 — Views

Um DBA cria uma **VIEW** que junta três tabelas e expõe **apenas algumas colunas** para um relatório.

Assinale a alternativa **correta** sobre views.

- A) Uma view **armazena fisicamente** uma cópia dos dados, ocupando o mesmo espaço das tabelas.
- B) Uma view é uma **consulta nomeada** (tabela virtual); simplifica o acesso e pode **restringir** colunas/linhas por segurança.
- C) Views **não podem** ser consultadas com `SELECT`.
- D) Views **sempre** aceitam `INSERT` / `UPDATE` sem restrição.

Q11 — Gabarito: B

Por que B: uma view (comum) é uma **consulta guardada** que se comporta como **tabela virtual** — os dados continuam nas tabelas-base. Serve para **abstrair, simplificar e proteger** (mostrar só parte dos dados).

- **A** — errada: quem armazena dados é a **view materializada**; a view comum **não** guarda cópia.
- **C** — errada: a view é consultada com `SELECT` normalmente.
- **D** — errada: views com **junção/agregação** geralmente **não** são atualizáveis.

Q12 — Indexação: hash × ordenado

Uma tabela terá **muitas buscas por igualdade** (procurar um valor exato) e também **buscas por faixa/ordenação** (períodos, listas ordenadas). A equipe compara índice **hash** e índice **ordenado (B-tree)**.

Assinale a alternativa **correta**.

- A) O índice hash é o melhor para buscas por faixa.
- B) Criar índice em todas as colunas é sempre vantajoso.
- C) Índices aceleram a escrita (`INSERT/UPDATE`), então devem cobrir tudo.
- D) O índice hash é ótimo para igualdade, mas ruim para faixa/ordenação; o B-tree atende bem os dois casos.

Q12 — Gabarito: D

Por que D: o **hash** localiza por **igualdade** em tempo quase constante, mas **não serve** para faixa/ordenação; o **B-tree** (ordenado/multinível) atende **igualdade e faixa**.

- **A** — errada: para **faixa**, o hash é justamente o **pior** caso.
- **B** — errada: **não se indexa tudo** — cada índice ocupa espaço e pesa na escrita.
- **C** — errada: índices **tornam a escrita mais lenta** (o índice também precisa ser mantido), não a aceleram.

 **Gabarito geral:** 1A · 2B · 3C · 4B · 5A · 6C · 7B · 8B · 9C · 10A · 11B · 12D

Bibliografia

- DATE, C. J. **Introdução a Sistemas de Banco de Dados**. 8ª ed. Elsevier, 2004.
- ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7ª ed. Pearson, 2019.
- SILBERSCHATZ, A.; KORTH, H.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7ª ed. Elsevier, 2020.

Dúvidas?

howard.cruz@faesa.br

Próxima unidade →

Unidade 4 — Modelagem Conceitual (ER)